

Module 07

LLMs + Anthropic SDK

From what a language model is to building production AI features

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is a Large Language Model?

A Large Language Model (LLM) is a neural network trained on vast amounts of text. It learns to predict what comes next in a sequence of words. When you send it a message, it generates a response one token at a time (a token is roughly 3/4 of a word). LLMs like Claude or GPT-4 are surprisingly good at following instructions, reasoning, summarising, translating, and writing code.

LLMs are **stateless** — they don't remember previous conversations unless you include them in the current request. The entire conversation history must be sent every time.

2. Tokens and Pricing

Pricing is per token (input + output). One token $\approx$ 0.75 words, or 4 characters. 'Hello, how are you?' is about 5 tokens. Code has more tokens per word because symbols and indentation each count.

```
# Estimate cost:
# Claude Sonnet 4.6 input: $3 per 1M tokens
# Claude Sonnet 4.6 output: $15 per 1M tokens

# 1000 API calls with:
#   average 500 input tokens = 500,000 tokens = $1.50
#   average 200 output tokens = 200,000 tokens = $3.00
# Total: $4.50 for 1000 calls
```

3. The Anthropic SDK — Basic Usage

```
pip install anthropic

from anthropic import Anthropic

client = Anthropic(api_key="YOUR_API_KEY")

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system="You are a helpful assistant for Indian finance topics.",
    messages=[
        {"role": "user", "content": "What is SEBI?"}
    ],
)

# Extract the text response
from anthropic.types import TextBlock
text = message.content[0].text if isinstance(message.content[0], TextBlock) else ""
print(text)

# Token usage
print(f"Input tokens: {message.usage.input_tokens}")
print(f"Output tokens: {message.usage.output_tokens}")
```

4. Multi-Turn Conversations

```

from anthropic import Anthropic

client = Anthropic(api_key="YOUR_API_KEY")

conversation = [] # keep history manually

def chat(user_message: str) -> str:
    conversation.append({"role": "user", "content": user_message})

    response = client.messages.create(
        model="claude-haiku-4-5-20251001",
        max_tokens=512,
        system="You are a concise assistant.",
        messages=conversation,
    )

    assistant_text = response.content[0].text
    conversation.append({"role": "assistant", "content": assistant_text})
    return assistant_text

# Multi-turn example
print(chat("My name is Himanshu"))
print(chat("What is my name?")) # correctly answers "Himanshu"
# (because the previous exchange is in conversation history)

```

5. Async Client — For FastAPI and asyncio

```

from anthropic import AsyncAnthropic
import asyncio

client = AsyncAnthropic(api_key="YOUR_API_KEY")

async def summarise(text: str) -> str:
    response = await client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=512,
        messages=[{
            "role": "user",
            "content": f"Summarise this in 3 bullets:\n\n{text}"
        }],
    )
    return response.content[0].text

async def main():
    # Run 3 summaries concurrently
    texts = ["Long article 1...", "Long article 2...", "Long article 3..."]
    summaries = await asyncio.gather(*[summarise(t) for t in texts])
    for s in summaries:
        print(s)

asyncio.run(main())

```

6. Structured Outputs — Getting JSON from the LLM

The most powerful pattern: ask the LLM to produce structured JSON. Use a clear schema in your prompt and parse the response.

```
import json
from anthropic import Anthropic

client = Anthropic(api_key="YOUR_API_KEY")

def score_article(title: str, summary: str) -> dict:
    prompt = f"""Score this article for content potential.
Article title: {title}
Summary: {summary}

Respond in JSON ONLY – no other text:
{{
  "score": <float 1.0-10.0>,
  "reasoning": "<why this score>",
  "angles": [<"angle 1">, "<angle 2">]
}}"""

    response = client.messages.create(
        model="claude-haiku-4-5-20251001",
        max_tokens=256,
        messages=[{"role": "user", "content": prompt}],
    )
    raw = response.content[0].text.strip()
    # Strip markdown code fences if present
    if raw.startswith("```"):
        raw = raw.split("```")[1]
        if raw.startswith("json"):
            raw = raw[4:]
    return json.loads(raw)

result = score_article("RBI raises rates by 25bps", "...")
print(result["score"], result["angles"])
```

Note: ContentAutomation2 uses this pattern in the scoring agent, summariser, and idea generator — the LLM returns structured JSON that is then validated with Pydantic.

7. Prompt Engineering Patterns

System prompt: sets the model's role, tone, constraints. Keep it stable across calls for caching benefits.

One-shot / Few-shot: give 1-3 examples of what you want. Far better than describing it in words.

```
system = """You summarise Indian financial news articles into 5 sections.
Always respond in this exact JSON format – no extra text.
```

Example output:

```
{
  "story_narrative": "2-3 sentences explaining what happened",
  "key_data_points": ["specific number or fact", "another fact"],
```

```

    "mechanism": "why this happened",
    "implications": "what this means for investors",
    "content_angles": ["angle 1", "angle 2"]
}"""

```

8. Batch API Calls with Semaphore

```

import asyncio
from anthropic import AsyncAnthropic

client = AsyncAnthropic(api_key="YOUR_API_KEY")
sem = asyncio.Semaphore(5)  # max 5 concurrent Claude calls

async def score(title: str) -> float:
    async with sem:
        resp = await client.messages.create(
            model="claude-haiku-4-5-20251001",
            max_tokens=64,
            messages=[{
                "role": "user",
                "content": f"Score this headline 1-10: {title}. Reply: {{\\"score\\": N}}"
            }],
        )
        data = json.loads(resp.content[0].text)
        return data["score"]

async def score_many(titles: list[str]) -> list[float]:
    return await asyncio.gather(*[score(t) for t in titles])

```

9. Token Cost Tracking

```

def calculate_cost(input_tokens: int, output_tokens: int, model: str) -> float:
    # Prices per million tokens (as of 2025)
    prices = {
        "claude-sonnet-4-6": {"in": 3.00, "out": 15.00},
        "claude-haiku-4-5-20251001": {"in": 0.80, "out": 4.00},
        "claude-opus-4-8": {"in": 15.00, "out": 75.00},
    }
    p = prices.get(model, {"in": 3.00, "out": 15.00})
    cost = (input_tokens * p["in"] + output_tokens * p["out"]) / 1_000_000
    return cost

```

10. Practice Exercises

- Exercise 1: Make your first Anthropic API call: ask 'What is compound interest?' and print the response.
- Exercise 2: Build a multi-turn chatbot that remembers the conversation for 5 turns.
- Exercise 3: Write a function that returns the summary of any article as structured JSON.
- Exercise 4: Use AsyncAnthropic and gather to score 5 headlines concurrently.
- Exercise 5: Add a Semaphore(3) and retry logic for rate limit errors.
- Exercise 6: Track input/output tokens across all calls and print total cost at the end.